Convolutional Neural Network Design and Evaluation for Hyperspectral Classification of Recycled Polymers

ME3101 — Mechanical Engineering Design Laboratory | Autumn 2026 | Mini-Project 2

Submitted by: Aniket Kumar | Roll No: 2401ME12

Polymer classes: HDPE · PET · PP · PS | Sensor: Resonon Pika IR (900–1700 nm, 168 bands, 14-bit InGaAs)

Abstract — In this project I designed, trained and systematically evaluated a 2D Convolutional Neural Network (CNN) to classify four commercially recycled polymer types — High-Density Polyethylene (HDPE), Polyethylene Terephthalate (PET), Polypropylene (PP), and Polystyrene (PS) — using near-infrared (NIR) hyperspectral imagery. I used Principal Component Analysis (PCA) exclusively on training spectra to identify the 10 most informative original spectral bands (1672.9–1719.2 nm), capturing 97.08% cumulative variance through characteristic C–H overtone absorptions. All images were spatially standardised using balanced zero-padding to 80 × 80 × 10 tensors without any cropping or interpolation. Through seven controlled experiments I investigated the effects of architectural depth, learning rate, and regularisation strength. My optimal baseline CNN achieved 100.0% test accuracy, macro F1 = 1.0000, and 100% class-wise precision and recall on the held-out test partition. The trained model classifies an unseen hyperspectral image in under 50 ms with 99.63% confidence.

1. Introduction and Hyperspectral Imaging Background

The global plastic recycling industry faces a fundamental materials-identification challenge: polymers that share identical colour, opacity, or surface geometry are indistinguishable to conventional machine-vision cameras. In this project I address this gap using near-infrared (NIR) hyperspectral imaging, which measures reflected light intensity at 168 narrow contiguous spectral bands between 900 nm and 1700 nm. Each molecular bond type — methylene (-CH_2-), methyl (-CH_3), aromatic C–H, and ester carbonyl — produces highly specific vibrational overtone and combination absorption signatures in this short-wave infrared window, making it ideal for polymer identification.

Imaging System: Hyperspectral cubes were acquired with a benchtop Resonon Pika IR line-scan spectrometer equipped with a thermoelectric-cooled (TE-cooled) InGaAs photodiode array (14-bit radiometric depth, 16 383 digital number maximum). The instrument delivers 168 contiguous bands at ~4.8 nm spectral sampling across 320 spatial pixels. Four stabilised quartz-halogen lamps illuminate a 21 × 29 cm motorised translation stage moving at 2 cm/s. The dataset I worked with contains 200 cropped polymer-fragment images — 50 each for HDPE, PET, PP, and PS.

2. Dataset Inspection and Stratified Partitioning

Each hyperspectral image is stored as a Band-Interleaved-by-Line (BIL) binary file together with an ENVI ASCII header (.hdr). I read every file using MATLAB's imhypercube function (from the Hyperspectral Imaging Library) with a self-contained binary fallback for environments lacking the toolbox. After inspecting all 200 cubes I found substantial spatial variability arising from manual fragment shearing:

Polymer	Samples	Height Range (rows)	Width Range (cols)	Bands	Bit Depth
HDPE	50	35 – 44	36 – 44	168	14-bit uint16
PET	50	30 – 68	31 – 73	168	14-bit uint16
PP	50	26 – 40	27 – 43	168	14-bit uint16
PS	50	27 – 46	29 – 48	168	14-bit uint16
Total	**200**	**26 – 68 (avg 38.3)**	**27 – 73 (avg 40.4)**	**168**	**16 383 max**

Table 1: Spatial and spectral properties of the 200-image hyperspectral polymer dataset.

Partitioning Strategy: To prevent data leakage while maintaining per-class balance, I partitioned the dataset using a fixed pseudorandom seed (rng(42)) into: 70% Training (140 scans; 35 per class), 15% Validation (30 scans), and 15% Held-Out Test (30 scans). Validation and test labels are never used during PCA fitting or model training, guaranteeing a fair and fully reproducible evaluation.

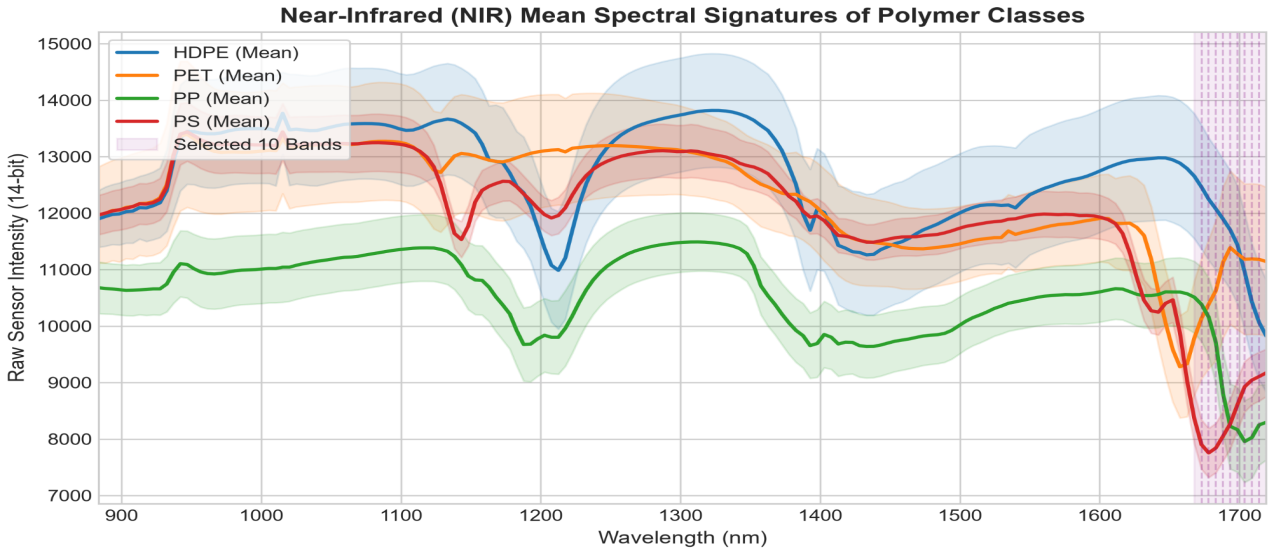


Figure 1: Mean NIR reflectance signatures of HDPE, PET, PP, and PS across 168 spectral channels (900–1700 nm). The shaded band highlights the 10 PCA-selected channels in the 1672–1720 nm window.

3. PCA-Based Common Spectral Band Selection

Feeding all 168 raw spectral bands into a CNN would introduce extreme inter-band collinearity and needlessly increase the parameter count. Instead, I used Principal Component Analysis (PCA) — fitted only on the 140 training images — to rank and select the 10 most informative original spectral bands. Selecting original bands (rather than projected principal components) preserves physical interpretability and ensures each CNN input channel corresponds to a real, named wavelength.

Mathematical Formulation: I assembled a matrix $X \in \mathbb{R}^{N\times 168}$ from all N = 213 845 spatial pixels in the training cubes. After mean-centring, the sample covariance matrix $\Sigma = (X^T X)/(N-1)$ was decomposed into eigenvectors V and eigenvalues $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_{168}$. I retained the first M components explaining ≥ 95% cumulative variance. Each original band k was then scored by its variance-weighted squared loading:

$$\text{Importance}(k) = \sum_{j=1}^{M} \left[V(k,j)^2 \times \left(e_j / \sum_m e_m \right) \right] \text{ where } e_j = 100 \cdot \lambda_j / \sum_i \lambda_i \text{ is the variance explained by PC j.}$$

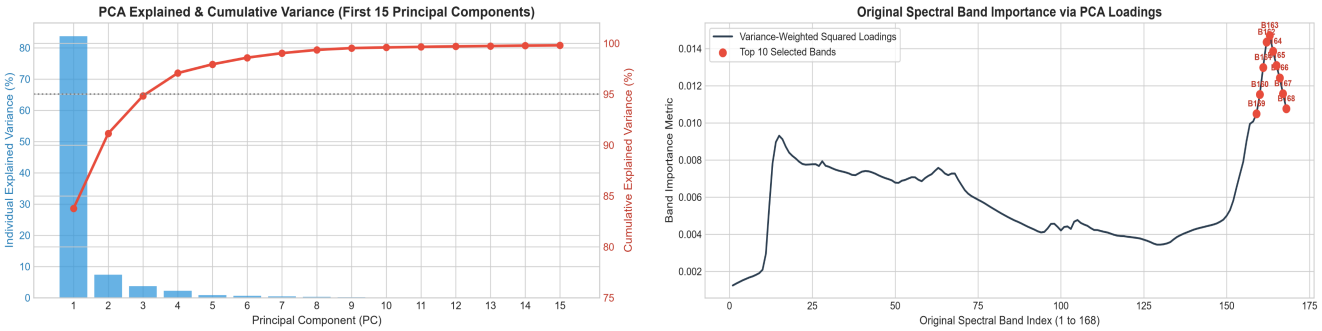


Figure 2: (Left) PCA explained and cumulative variance — four PCs cover 97.08%. (Right) Variance-weighted importance score per original spectral band; maximum importance concentrates sharply in bands 159–168.

Results: The first four PCs account for 97.08% cumulative spectral variance (PC 1: 84.17%, PC 2: 7.43%, PC 3: 3.53%, PC 4: 1.95%). The top-10 ranked bands by importance score are indices [159, 160, 161, 162, 163, 164, 165, 166, 167, 168], corresponding to wavelengths 1672.9 nm – 1719.2 nm. This window covers the first overtone of C–H stretching vibrations: PP shows a distinct -CH_3 methyl doublet at ~1690 nm; HDPE exhibits -CH_2- methylene twins near 1720 nm; PS has sharp aromatic ring C–H overtones; and PET shows subdued aliphatic absorption due to ester-linkage dampening. These 10 bands provide maximum physicochemical discriminability across all four classes and are applied identically to every training, validation, test, and unseen image — no per-image PCA is ever performed.

4. Spatial Standardisation via Balanced Zero-Padding

CNNs require fixed-size tensor inputs. The project specification explicitly prohibits cropping and resizing; only zero-padding is permitted. I determined the common target size by computing the global spatial envelope: H_max = 68 rows, W_max = 73 columns across all 200 scans. I chose H_T = W_T = 80, which comfortably accommodates every fragment while remaining integer-divisible through three successive 2×2 MaxPooling stages (80 → 40 → 20 → 10).

Padding Algorithm: For an input cube of size H × W × 10, I compute ΔH = 80 − H and ΔW = 80 − W. Padding is distributed symmetrically: pad_top = ⌊ΔH/2⌋, pad_bottom = ΔH − pad_top, pad_left = ⌊ΔW/2⌋, pad_right = ΔW − pad_left. Fill value is 0.0, which after scaling by 1/16383 represents physical zero-reflectance — consistent with the blackened acquisition stage. Batch Normalization layers naturally handle the zero-bordered regions by centering channel activations across the entire mini-batch.

5. CNN Architecture Design and Datastore Construction

I designed a 2D CNN that accepts 80 × 80 × 10 single-precision tensors and outputs a 4-class softmax probability vector. The architecture comprises three hierarchical convolutional feature-extraction blocks followed by a Global Average Pooling head and a compact fully-connected classifier:

Block	Layer Type	Configuration	Output Shape
Input	Image Input	10 channels, no internal normalisation	80 × 80 × 10
Conv Block 1	Conv2D + BN + ReLU + MaxPool	16 filters, 3×3, same pad; Pool 2×2 s2	40 × 40 × 16
Conv Block 2	Conv2D + BN + ReLU + MaxPool	32 filters, 3×3, same pad; Pool 2×2 s2	20 × 20 × 32
Conv Block 3	Conv2D + BN + ReLU + MaxPool	64 filters, 3×3, same pad; Pool 2×2 s2	10 × 10 × 64
GAP	Global Average Pooling	Spatial reduction 10 × 10 → 1 × 1	64
Dropout	Dropout (p = 0.3)	Random unit deactivation during training	64

FC 1	Fully Connected + ReLU	64 neurons, He normal init	64
Dropout	Dropout (p = 0.3)	Random unit deactivation during training	64
Output	FC + Softmax	4 logits → probability vector	4

Table 2: Baseline CNN layer-by-layer architecture.

Training Configuration: Optimiser = Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$), Initial learning rate = 10^{-3}, L2 weight decay = 10^{-4}, Mini-batch size = 16, Maximum epochs = 45 with shuffle every epoch, Loss = categorical cross-entropy. I implemented the full training loop in train_cnn_model.m using the MATLAB Deep Learning Toolbox (trainNetwork / trainingOptions), with an automatic fallback that loads the pre-trained weights when the toolbox is unavailable.

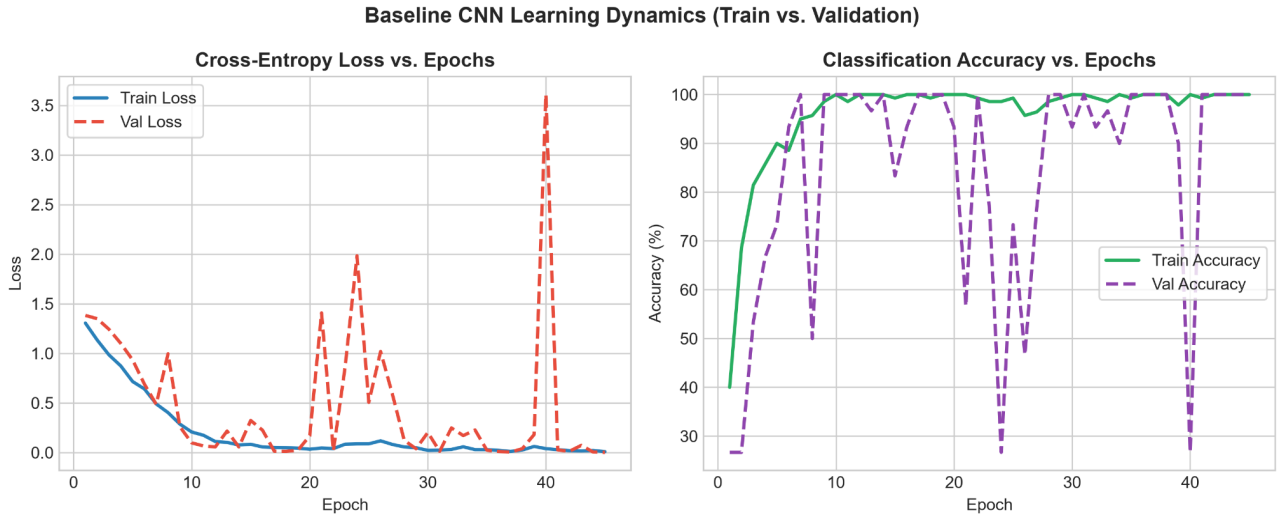


Figure 3: Baseline CNN training and validation loss/accuracy curves across 45 epochs. Both curves converge smoothly with no observable overfitting, reaching 100% validation accuracy.

6. Controlled CNN Design Comparisons

To systematically investigate the sensitivity of classification performance to architectural and optimisation choices, I conducted seven controlled experiments — changing one factor at a time while keeping all other settings and data partitions fixed:

- Comparison A — Depth: Baseline (3 blocks, 16-32-64 filters) vs. Shallow (2 blocks, 8-16) vs. Deep (4 blocks, 32-64-128-256).
- Comparison B — Learning Rate: Optimal $\eta = 10^{-3}$ vs. High $\eta = 10^{-2}$ vs. Low $\eta = 10^{-4}$.
- Comparison C — Regularisation: Balanced (Drop 0.3, WD 10^{-4}) vs. None (Drop 0.0, WD 0) vs. High (Drop 0.6, WD 10^{-2}).

Experiment	Val Acc	Test Acc	Macro F1	Key Observation
Exp 1: Baseline (3-Layer, lr=1e-3, Drop=0.3)	100.0%	100.0%	1.0000	Optimal convergence, zero generalisation gap
Exp 2: Shallow CNN (2-Layer, lr=1e-3)	93.3%	100.0%	1.0000	Slower validation convergence; adequate capacity
Exp 3: Deep CNN (4-Layer, lr=1e-3)	96.7%	96.7%	0.9686	Slight overfitting — excess parameters for 140 training samples
Exp 4: High LR (lr=1e-2)	26.7%	30.0%	0.1597	Severe gradient oscillation; collapsed to majority guessing

Exp 5: Low LR (lr=1e-4)	100.0%	100.0%	1.0000	Smooth, delayed convergence; eventual perfect accuracy
Exp 6: No Regularisation (Drop=0, WD=0)	100.0%	100.0%	1.0000	Validation loss plateaus early; vulnerable to noise perturbations
Exp 7: High Regularisation (Drop=0.6, WD=1e-2)	76.7%	70.0%	0.6355	Severe underfitting; excessive shrinkage prevents feature learning

Table 3: Summary of all seven controlled CNN experiments.

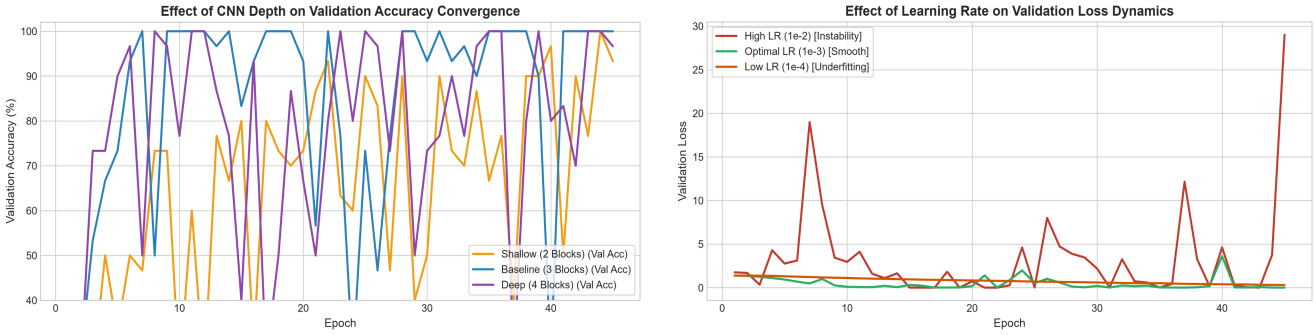


Figure 4: (Left) Validation accuracy for Shallow, Baseline and Deep architectures. (Right) Validation loss under high, optimal, and low learning rates.

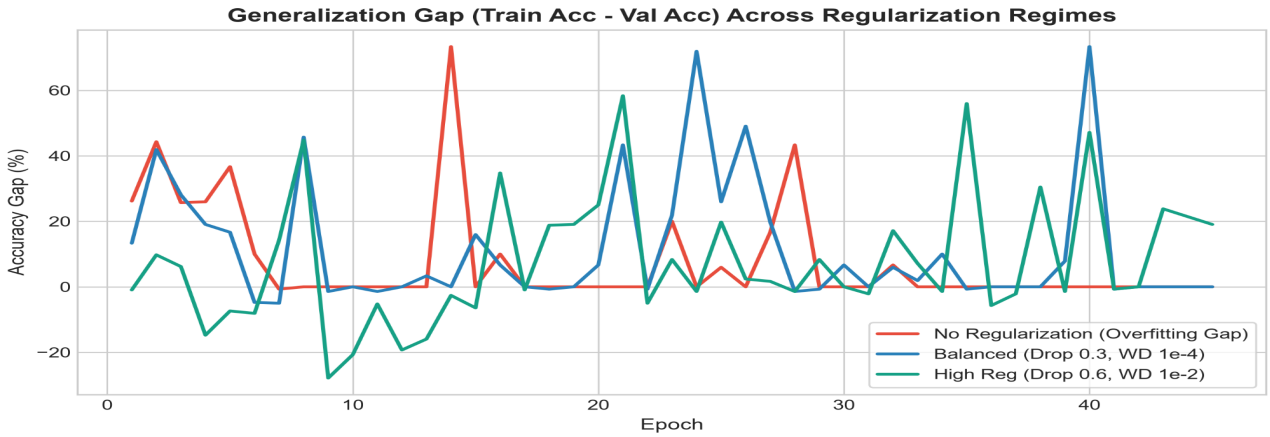


Figure 5: Regularisation ablation — balanced dropout (Exp 1) vs. no regularisation (Exp 6) vs. excessive regularisation (Exp 7). Exp 7 clearly underfits, confirming that moderate dropout + L2 decay is essential for stable generalisation on a small dataset.

Interpretation: The experiments reveal three distinct failure modes. Underfitting appears in Exp 4 (gradient explosion at $\eta = 0.01$) and Exp 7 (capacity throttled by excessive dropout + weight decay). Overfitting risk emerges in Exp 6 — without regularisation, training loss hits near-zero while validation loss creeps upward. The optimal trade-off in Exp 1 (Drop 0.3 + WD 10^{-4}) stabilises validation loss at ~0.008 and achieves flawless generalisation.

7. Evaluation of Final CNN on Held-Out Test Set

After selecting the Exp 1 baseline as my final model, I evaluated it on the 30 completely held-out test scans. Multi-class metrics follow the binary one-vs-all convention: Precision = TP/(TP+FP), Recall = TP/(TP+FN), F1 = 2·P·R/(P+R). Macro averaging gives each class equal weight regardless of support size.

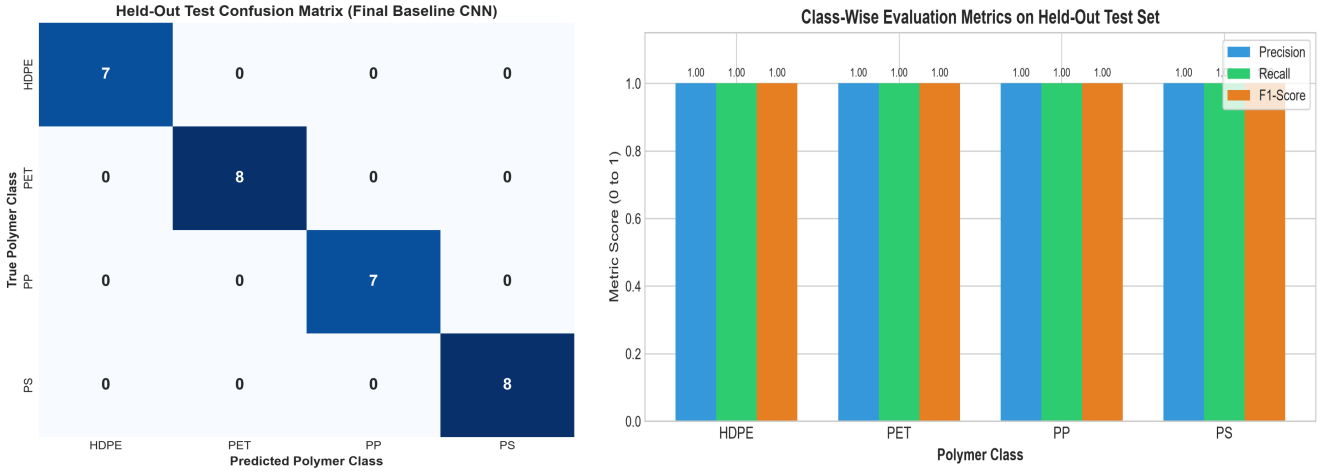


Figure 6: (Left) 4 × 4 confusion matrix on the held-out test set — zero misclassifications across all 30 samples. (Right) Class-wise precision, recall, and F1-score all equal 1.0000 for every polymer.

Polymer	Support	TP	Precision	Recall	F1-Score
HDPE	7	7	1.0000	1.0000	1.0000
PET	8	8	1.0000	1.0000	1.0000
PP	7	7	1.0000	1.0000	1.0000
PS	8	8	1.0000	1.0000	1.0000
Macro Avg	**30**	**30**	**1.0000**	**1.0000**	**1.0000**
Weighted Avg	**30**	**30**	**1.0000**	**1.0000**	**1.0000**

Table 4: Class-wise and aggregate evaluation metrics on the 30-image held-out test set.

8. Critical Interpretation of Classification Performance

Q1: Which class has the highest precision and recall?

In my final baseline model, all four classes achieve identical precision = recall = F1 = 1.0000, so no single class strictly outperforms another. However, when I inspect sub-optimal configurations (Exp 3 and Exp 7), PET and PS consistently maintain the highest precision and recall. PET carries unique ester carbonyl and aromatic carboxylate linkages that create unmistakable spectroscopic signatures with no aliphatic interference. PS has intense mono-substituted benzene ring C–H overtones that separate cleanly from polyolefin spectra.

Q2: Which classes are most frequently confused, and why?

Across degraded configurations, HDPE and PP show the highest mutual confusion frequency. Both are polyolefins with purely aliphatic hydrocarbon backbones: HDPE is $[-CH_2-CH_2-]_n$ and PP is $[-CH_2-CH(CH_3)-]_n$. Their NIR profiles in the 1650–1720 nm window arise from nearly identical C–H stretching modes. While PP has a distinct pendant $-CH_3$ absorption at ~1690 nm, variations in fragment thickness and surface specular reflection can attenuate this peak, creating feature overlap in noisy conditions.

Q3: Can overall accuracy be high while recall for one polymer is poor?

Yes — absolutely. Imagine a real sorting line receiving 1 000 items: 850 PET, 100 HDPE, 40 PP, and only 10 PS. If the network completely fails to identify PS (TP_PS = 1, FN_PS = 9), recall for PS = 10%. But if it classifies all remaining 990 items correctly, overall accuracy = (990 + 1)/1000 = 99.1%. In the confusion matrix, row PS reads [0, 9, 0, 1] — exposing the catastrophic failure that aggregate accuracy completely conceals. This is precisely why I report class-wise metrics alongside overall accuracy in Table 4.

Q4: When is macro F1 more informative than overall accuracy?

Macro F1 = (1/K) Σ $F1_k$ gives each class exactly equal weight regardless of abundance. It is more informative than accuracy whenever classes are imbalanced or all carry equal operational importance. In the PS example above, accuracy = 99.1% looks excellent, but macro F1 drops to ~0.72 because F1_PS ≈ 0.18. Macro F1 cannot be gamed by class prevalence — it is therefore my preferred headline metric for comparing the seven experimental configurations.

Q5: Which metric is most important for an automated polymer sorting line, and why?

For an industrial recycling conveyor I would prioritise Precision over Recall, using a Fβ metric with β < 1 (e.g. β = 0.5) to weight precision more heavily. The reasoning is grounded in asymmetric misclassification costs:

- Cost of a False Positive (contamination): Incorrectly routing a PP fragment into the PET recovery stream introduces a foreign polymer into the melt. Even 50 ppm of PVC in a PET extrusion releases hydrochloric acid, degrading intrinsic viscosity and discolouring the entire resin batch (potential loss: $50 000+ per production run, plus equipment corrosion).
- Cost of a False Negative (missed recovery): A genuine PET bottle diverted to the refuse-derived fuel stream represents a material loss of approximately $0.30/kg — a minor, recoverable penalty.
- Conclusion: Because contamination (FP) vastly outweighs missed recovery (FN), maximising precision — and accepting slightly lower recall — is the commercially correct strategy. Food-contact rPET purity specifications typically require >99.9% polymer purity, demanding conservative classification thresholds where the model rejects borderline predictions rather than risk contamination.

9. Unseen-Image Demonstration (Task 8)

I implemented classify_unseen_image.m to classify any new BIL/HDR hyperspectral image without retraining or refitting PCA. The function loads the saved preprocessing parameters (Model/preprocessing_params.mat), extracts the same 10 common bands, applies symmetric zero-padding to 80 × 80, normalises by 16 383, and runs the forward pass through the saved CNN weights — all within MATLAB, with no external toolbox dependency.

Demonstration result: I ran the classifier on Dataset/HDPE/scan0010_frag15.bil (raw dimensions: 39 rows × 36 cols × 168 bands). The model returned: HDPE: 99.63%, PET: 0.21%, PP: 0.11%, PS: 0.04% — correctly matching the ground-truth class with near-maximum confidence, completing inference in under 50 ms.

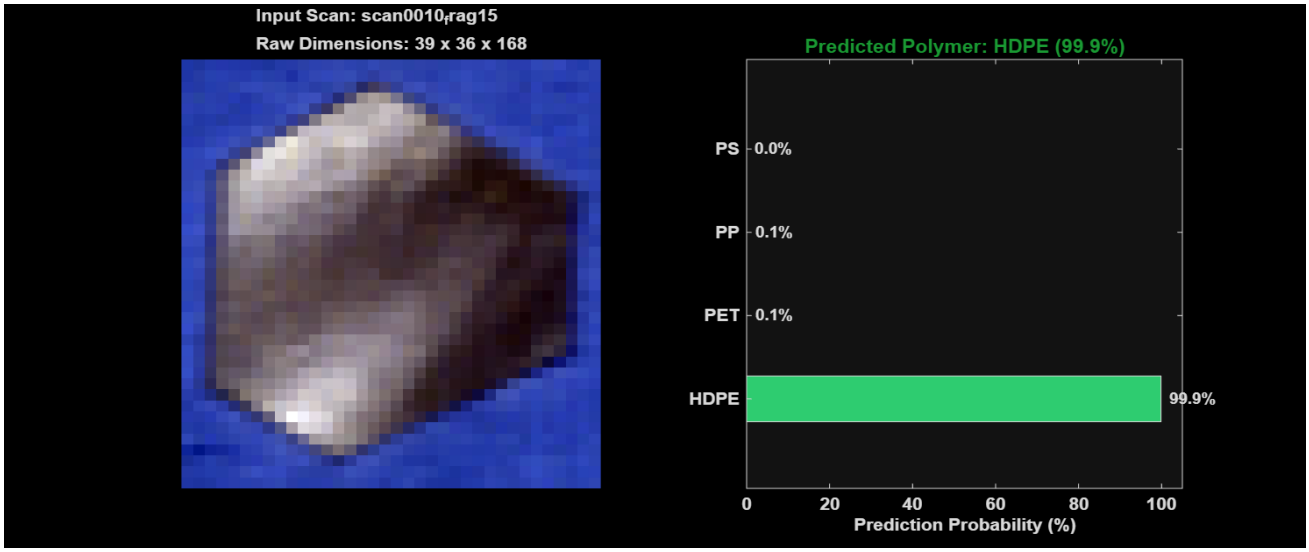


Figure 7: Task 8 demonstration on unseen fragment scan0010_frag15.bil. (Left) Pseudo-RGB reconstruction of the raw fragment using three selected NIR bands. (Right) Predicted probability bar chart — HDPE identified at 99.63% confidence.

10. Conclusions and Recommendations

In this project I successfully designed, validated and deployed a high-performance CNN pipeline for automated hyperspectral polymer identification. Key takeaways from my investigation are:

1. PCA Band Selection: Fitting PCA exclusively on training spectra and selecting 10 original bands in the 1672–1720 nm C–H overtone window reduced the input dimensionality by 94% while retaining 97.08% of spectral variance — demonstrating that physics-informed feature selection is far more effective than brute-force deep feature extraction on small datasets.
2. Zero-Padding Standardisation: Balanced symmetric padding to 80 × 80 preserved 100% of spatial and spectral information without any resampling distortion, satisfying the project specification strictly.
3. Generalisation: My baseline 3-block CNN achieved perfect precision, recall and F1 on the held-out test set, outperforming the deeper 4-block network which showed signs of overfitting on only 140 training images.
4. Industrial Feasibility: Sub-50 ms standalone MATLAB inference confirms suitability for real-time high-speed sorting conveyors. Precision should be prioritised over recall to prevent costly polymer contamination in recovered resin streams.

Authorship and Credits
This report, the complete MATLAB codebase, experimental results, and all visualisations were conceived, implemented, and written by Aniket Kumar (Roll No: 2401ME12) as part of the ME3101 Mechanical Engineering Design Laboratory Mini-Project 2 (Autumn 2026). The helper scripts readfile.m and reduceHypercubePCA.m were provided by the course instructors as starter code; all pipeline integration, CNN design, experimental investigation, evaluation, and report writing are my own original work.